

Bienvenid@s a...

UNIVERSIDAD
NACIONAL DE
HURLINGHAM



Programación con Objetos II

2026 - Primer cuatrimestre

Semana 6

Manejo de Excepciones

Equipo docente



Dario
Moreira



Hernan
Coniglio



Mauricio
Pintos



Gabriel
Ledezma



Pablo
Isrelsky



Mr. X



PROGRAMACIÓN

ORIENTADA

OBJETOS

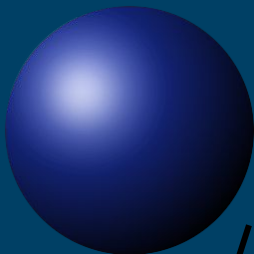




emisor



mensaje



receptor



```
import java.io.*;
import java.net.*;
import java.security.*;
import protection;

public class Client {
    public void sendAuthentication(String user,
    OutputStream outstream) throws IOException {
        DataOutputStream out = new DataOutputStream(outstream);
        long t1 = (new Date()).getTime();
        double q1 = Math.random();
        byte[] protected1 = Protection.m
        long t2 = (new Date()).getTime();
        double q2 = Math.random();
        byte[] protected2 = Protection.m
        out.writeUTF(user);
        out.writeInt(protected1.length);
        out.write(protected2);
        out.flush();
    }
}

public static void main(String[] args) {
    String host = args[0];
    int port = 999;
    String user = "John";
    String password = "Sh
    Socket s = new Socket

    Client client = new
    client.sendAuthent
```

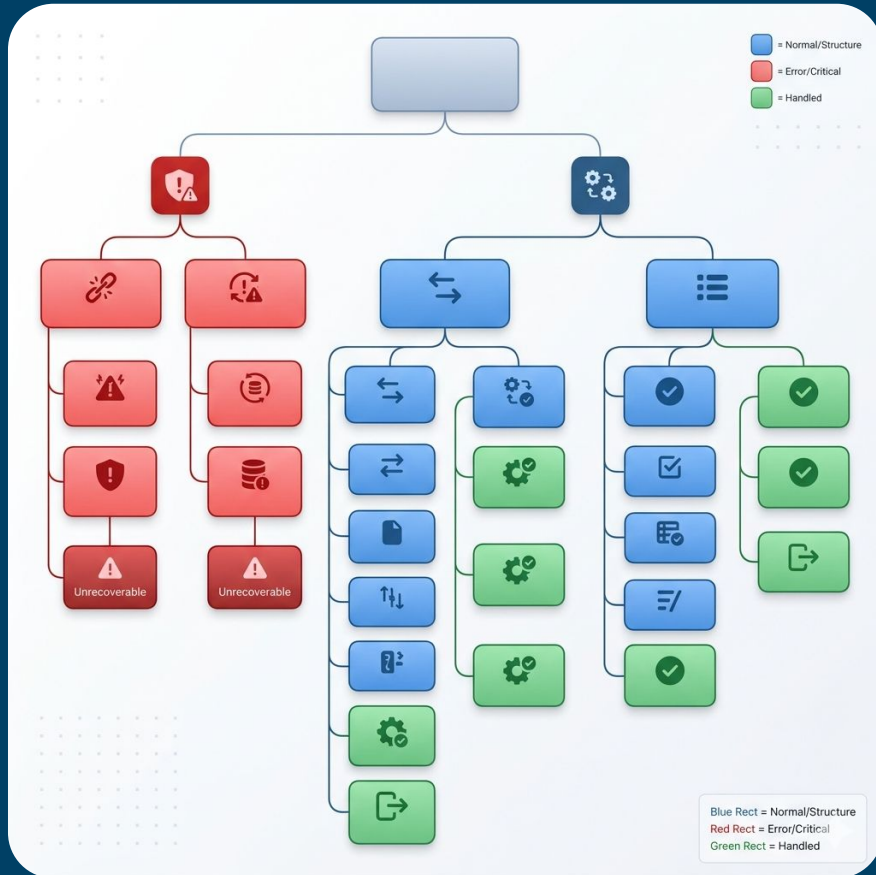
Introducción al Manejo de Excepciones

¿Qué es una Excepción?



- Un programa es un conjunto de objetos que se envían mensajes.
- Una excepción es un evento que altera el flujo normal de ejecución.
- Representa una situación anormal o inesperada.
- **Ejemplos:**
 - ◆ División por cero
 - ◆ Acceso fuera de rango
 - ◆ Referencia nula





Jerarquías y tipos de Excepciones



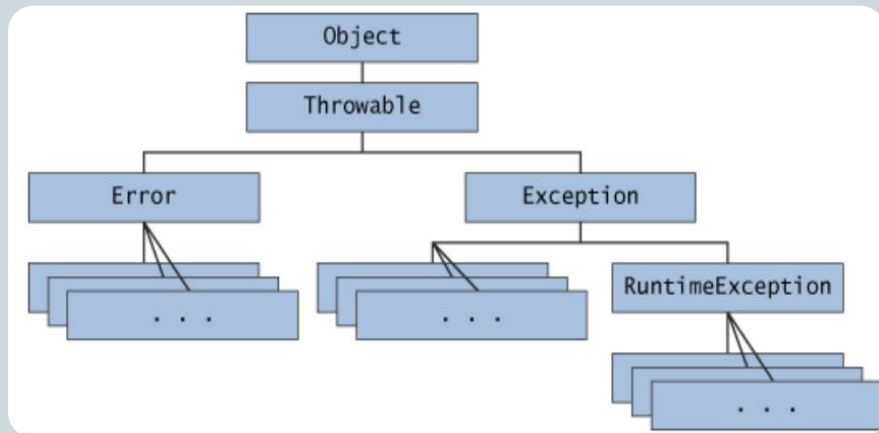
Jerarquía de Excepciones en Java

→ Throwable (raíz)

- ◆ Error → fallas graves (no manejables)
- ◆ Exception → errores manejables
 - RuntimeException → no chequeadas

→ Ejemplos:

- ◆ NullPointerException
- ◆ IOException



Excepciones: Negocio vs Programa

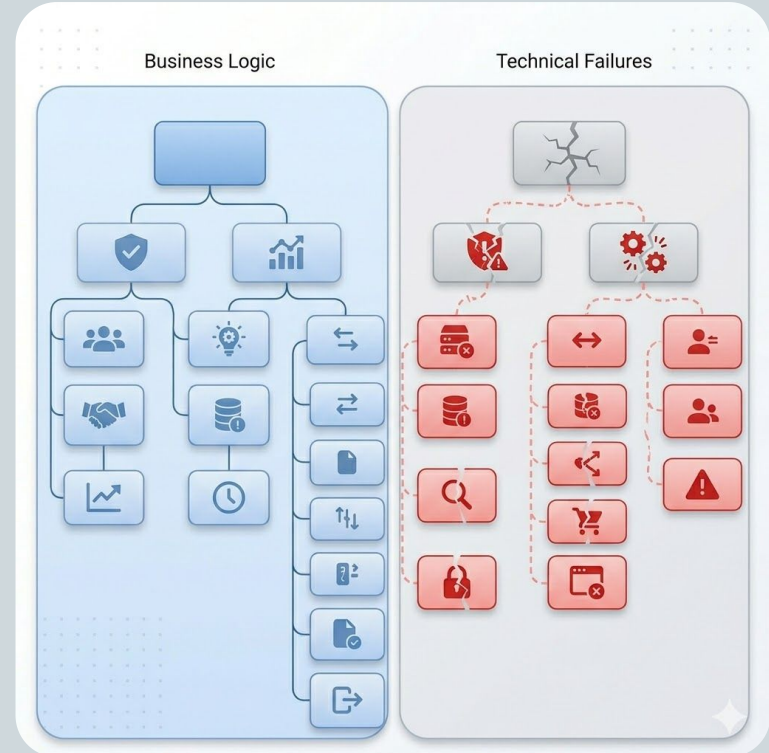


→ De negocio

- ◆ Parte del dominio
- ◆ Esperables
- ◆ Ej: “saldo insuficiente”

→ De programa

- ◆ Técnicas
- ◆ No esperadas
- ◆ Ej: error de conexión, null





Excepciones: Checked vs Unchecked

→ Checked (Exception)

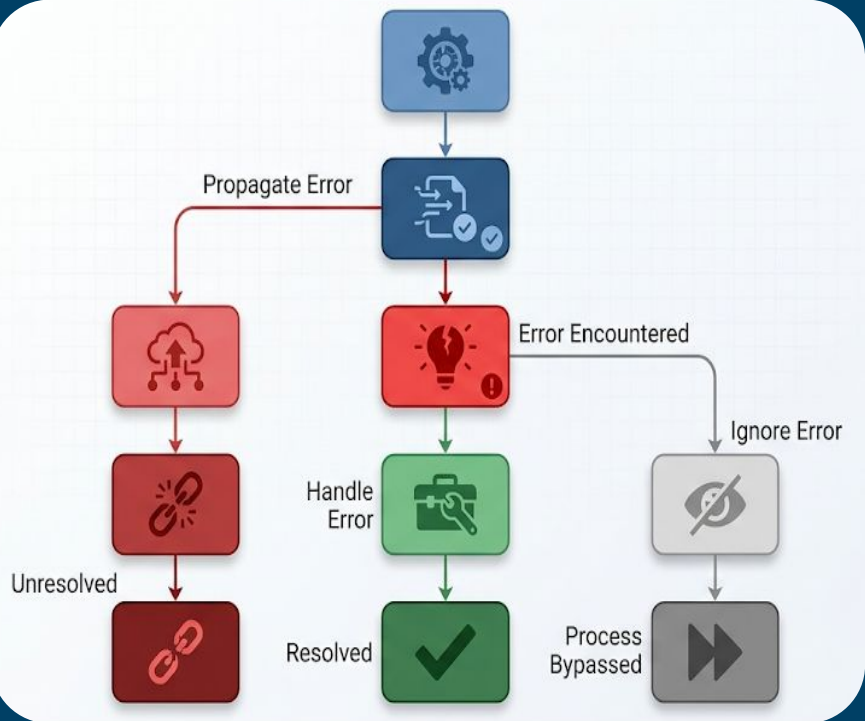
- ◆ Obligan a declarar o capturar

→ Unchecked (RuntimeException)

- ◆ No obligatorias

→ Problema de las checked:

- ◆ Rompen polimorfismo
- ◆ Ensucian código



¿Cómo manejamos las Excepciones?



Lanzar excepciones

→ Al usar throw:

```
void procesarArchivo(String path) throws MiExcepcion {  
    if (path == null) {  
        throw new MiExcepcion("El path no puede ser null");  
    }  
    abrirArchivo(path);  
    hacerAlgo();  
}
```



Crear excepciones

→ Al crear clases propias:

- ◆ Extender RuntimeException (recomendado)

→ Mensajes:

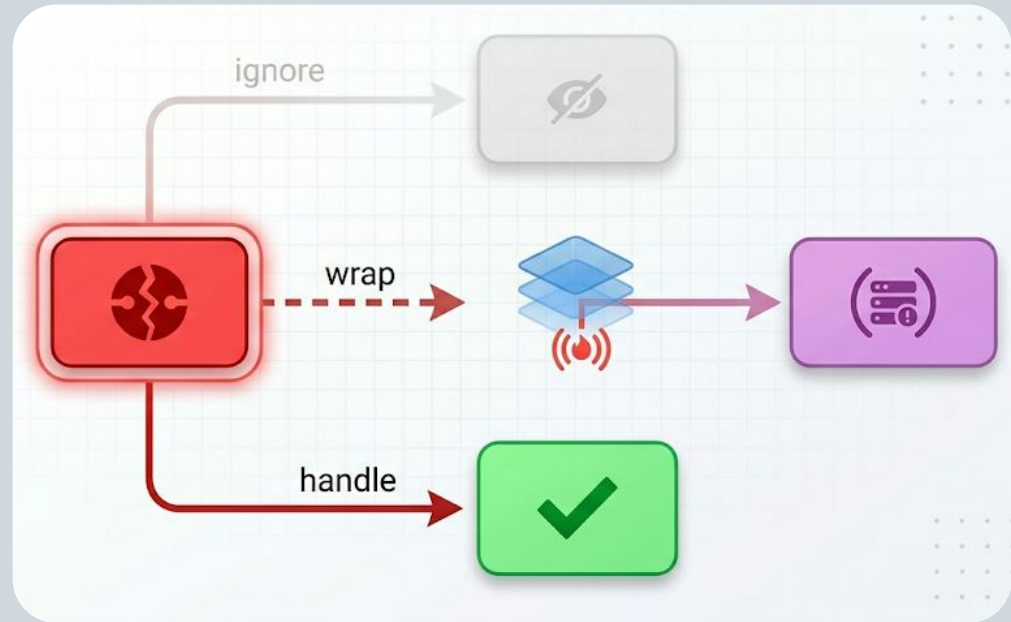
- ◆ Claros
- ◆ Orientados a solución

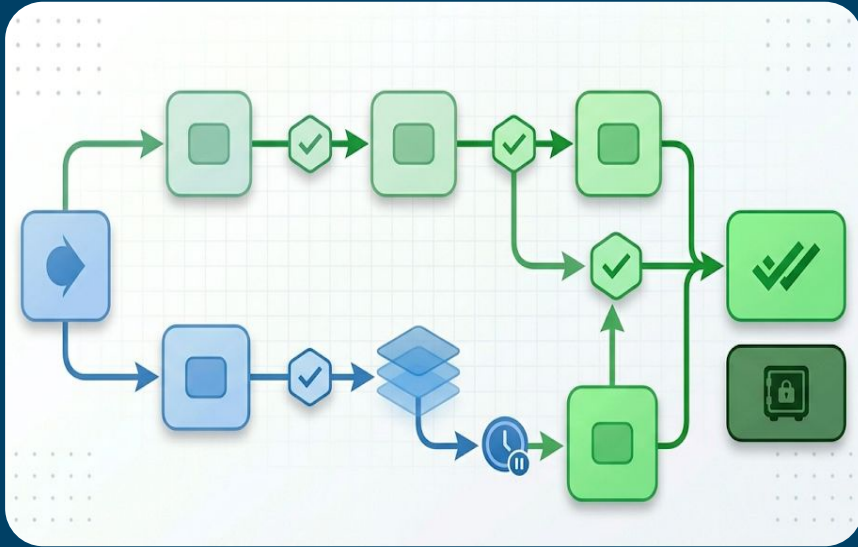
```
class MiExcepcion extends RuntimeException {  
  
    public MiExcepcion(String mensaje) {  
        super(mensaje);  
    }  
}
```



Estrategias ante una excepción

- No tratarla
 - ◆ Se propaga
- Envolverla
 - ◆ Traducir a otra excepción
- Tratarla
 - ◆ Acción correctiva real





Controlando el flujo de ejecución



El problema del flujo tradicional

- Sin excepciones, se usan múltiples if/else.
- Cada paso valida posibles errores.
- Problemas:
 - ◆ Código difícil de leer
 - ◆ Lógica de negocio mezclada con validaciones
 - ◆ Baja mantenibilidad

```
abrirArchivo();  
if (ocurrióError()) {  
    // manejo del error  
    // mezclado con la lógica  
    loggearError();  
    cerrarArchivo();  
    return;  
}  
hacerAlgo();  
if (otroError()) {  
    loggearError();  
    cerrarArchivo();  
    return;  
}  
cerrarArchivo();
```



La solución: Try-Catch

→ Separa:

- ◆ Flujo normal → try
- ◆ Manejo de errores → catch

→ Mejora:

- ◆ Legibilidad
- ◆ Organización

→ Permite enfocar el código en el objetivo principal

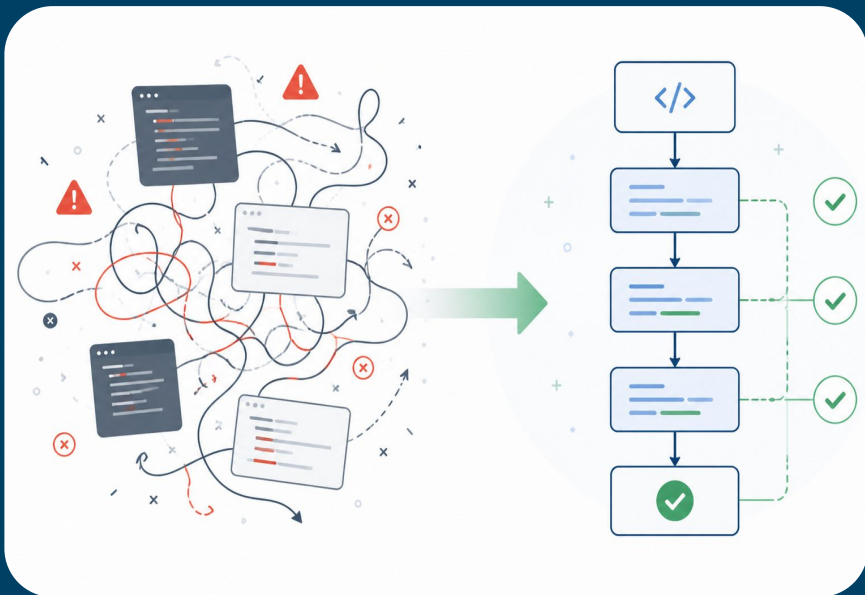
```
abrirArchivo();  
try {  
    hacerAlgo();  
} catch (Exception ex) {  
    manejarError(ex);  
}  
cerrarArchivo();
```



Bloque Finally

```
abrirArchivo();  
try {  
    hacerAlgo();  
} catch (Exception ex) {  
    manejarError(ex);  
} finally {  
    cerrarArchivo();  
}
```

- Se ejecuta siempre
- Uso:
 - ◆ Liberar recursos
 - ◆ Cerrar conexiones
- Una alternativa moderna:
(si quieren investigar)
 - ◆ try-with-resources



**Buenas prácticas
y conclusión**



Buenas prácticas

- ✓ Capturar sólo lo necesario
- ✗ No dejar catch vacío
- ✗ No usar excepciones como lógica
- ✓ Mensajes claros

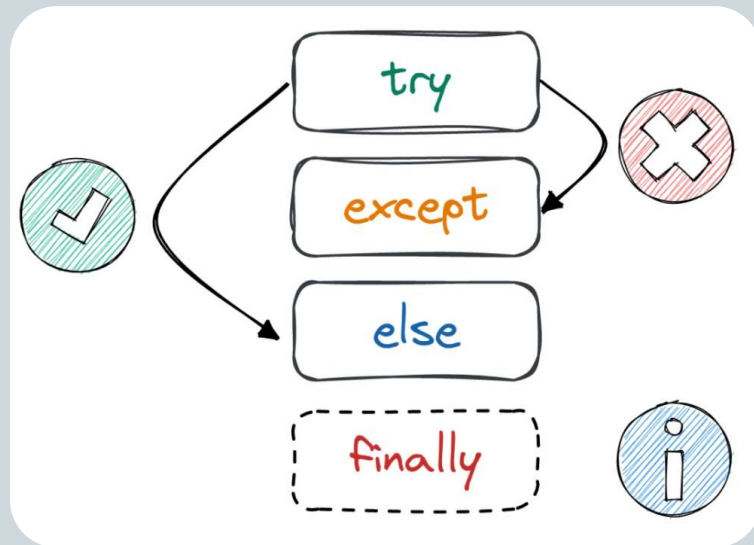
✗ Ejemplo incorrecto

```
catch(Exception e) {  
    e.printStackTrace();  
}
```



Conclusión

- Las excepciones son una herramienta de diseño
- Permiten:
 - ◆ Separar responsabilidades
 - ◆ Mantener código limpio
- Mal uso → código frágil
- Buen uso → código mantenible



Consultar el material completo: [Intro a manejo de errores con excepciones en Java](#)



Educación pública, gratuita y de calidad